

Problem Set 02
Enero 2026

Datasets (ver anexo).

- **Parte 1 y 2:** retail_reviews.csv
- **Parte 3:** video_ratings.csv
- **Parte 4:** music_logs.csv
- **Parte 5:** movie_metadata.csv

Nota: La data contiene ruido intencional (valores nulos, textos sucios, sparsity extrema) para simular entornos reales.

Parte 1. Procesamiento de Lenguaje Natural (NLP) y Embeddings

1.1. Análisis de Embeddings y Álgebra Lineal

Utilizando el dataset `retail_reviews.csv`, entrena un modelo word2vec (o utiliza uno pre-entrenado como GloVe/FastText) sobre el corpus de reseñas.

- **a)** Encuentra los 5 términos más similares semánticamente a la palabra "defectuoso" y "rápido" utilizando la similitud coseno.
- **b) Interpretación Matemática:** Explica por qué utilizamos la **Similitud Coseno** en lugar del **Producto Punto** estándar para medir similitud semántica en este contexto. Basa tu respuesta en la relación entre la magnitud del vector (norma L2) y la frecuencia del término en el corpus.
- **c)** Realiza una operación de álgebra vectorial análoga a "Rey - Hombre + Mujer = Reina" pero aplicada al contexto de retail (ej: "Smartphone - Pantalla + Batería" o similar). Comenta si el resultado tiene sentido semántico.

1.2. Clasificación de Texto con Transformers

Entrena un modelo de clasificación (puedes usar una regresión logística simple con features TF-IDF como baseline y luego comparar con un modelo basado en BERT o RoBERTa fine-tuneado o usando sus embeddings) para predecir la columna sentiment (Positivo/Negativo).

- **Reporte:** Compara el **F1-Score** de ambos enfoques. ¿En qué casos específicos falla el modelo TF-IDF donde el modelo de Transformers acierta?.

Parte 2. Topic Modeling (No supervisado)

2.1. Implementación de BERTopic

Utilizando las reseñas sin etiqueta del dataset `retail_reviews.csv`, implementa un pipeline de BERTopic para descubrir los principales temas de conversación de los clientes.

- Debes explicitar y justificar la configuración de los tres pasos clave del pipeline:
 1. **Embeddings:** ¿Qué modelo de sentencia utilizaste? (ej: `all-MiniLM-L6-v2`). Justifica tu decisión.
 2. **Reducción de Dimensionalidad (UMAP):** ¿Por qué es necesario reducir dimensiones antes de clusterizar?
 3. **Clusterización (HDBSCAN):** ¿Cómo maneja este algoritmo el "ruido" o los documentos que no encajan en ningún tópico?

2.2. Interpretación de Negocio

Visualiza los tópicos resultantes y genera una tabla con el "Top 5 Tópicos" y sus palabras clave (c-TF-IDF). Identifica si existe algún "tópico latente" relacionado con problemas logísticos (ej: envíos tardíos) que no sea evidente en un análisis de conteo de palabras simple.

Parte 3. Sistemas de Recomendación: Filtrado Colaborativo Explícito

3.1. Factorización Matricial (SVD)

Utilizando el dataset `video_ratings.csv` (UserID, MovieID, Rating 1-5):

- Implementa el algoritmo **SVD (Funk-SVD)** utilizando la librería `Surprise`.
- Realiza una validación cruzada (5-fold) y reporta el **RMSE** promedio.
- **Pregunta Teórica:** Si el RMSE es bajo (ej: 0.8 estrellas), ¿significa necesariamente que el sistema es bueno recomendando? Justifica tu respuesta contrastando el error de predicción vs. la calidad del ranking (Top-N).

3.2. El problema del Cold-Start

Identifica a los usuarios en el conjunto de test que tienen menos de 3 interacciones ("Usuarios fríos").

- Calcula el RMSE específico para este subgrupo.
- Propón y describe teóricamente (no es necesario programar) una estrategia **Híbrida** o basada en **Contenido** para mejorar el desempeño en estos usuarios.

Parte 4. Sistemas de Recomendación: Feedback Implícito y LTR

4.1. Mínimos Cuadrados Alternados (ALS)

Utilizando el dataset `music_logs.csv` (que contiene `play_count` en lugar de ratings explícitos):

- Implementa un modelo **ALS (Alternating Least Squares)** usando la librería `implicit`.
- Explica el concepto de "**Confianza**" (c_{ui}) en la función de costo de ALS. ¿Por qué un usuario que ha escuchado 0 veces una canción no tiene una "preferencia negativa" explícita, sino una confianza baja?

4.2. Evaluación de Ranking (NDCG)

Para un usuario aleatorio, genera el Top-10 de recomendaciones. Supón que conocemos la "verdad real" (ground truth) de lo que el usuario efectivamente escuchó la semana siguiente (test set).

- Calcula manualmente (o con código) el **NDCG@5** para esa recomendación.
- Explica cómo el NDCG penaliza al algoritmo si coloca una canción muy relevante en la posición 5 en lugar de la posición 1, a diferencia de la métrica **Precision@K**.

Parte 5. Desafíos avanzados

5.1. NLP: Análisis de Sentimiento Basado en Aspectos (ABSA) y Señales Mixtas

Los modelos de clasificación binaria (Positivo/Negativo) suelen fallar cuando una reseña contiene opiniones encontradas (ej: "El producto es excelente, pero la entrega tardó dos semanas").

- Filtra del dataset `retail_reviews.csv` aquellas reseñas que contengan al menos una palabra del vocabulario positivo Y una del negativo simultáneamente.
- Para este subconjunto "mixto", compara la etiqueta `sentiment` original (asignada por el modelo o ground truth) versus la realidad del texto.
- Si tuviéramos que automatizar la creación de tickets de soporte (CRM), ¿por qué un modelo de sentimiento global sería peligroso en estos casos? Propón una lógica algorítmica (pseudocódigo) para detectar específicamente problemas de "Envío" independientemente de la calidad del producto.

5.2. RecSys: Evaluación de Sesgo de Popularidad y "Long Tail"

Un sistema de recomendación con alto accuracy (bajo RMSE o alto Precision) suele sufrir de sesgo de popularidad: recomienda siempre los "Best Sellers", ignorando el catálogo de nicho (Long Tail).

- Usando el modelo entrenado en la Parte 3 (SVD) o 4 (ALS), genera recomendaciones Top-10 para todos los usuarios de test.

- Calcula la Cobertura de Catálogo (Catalog Coverage): ¿Qué porcentaje del total de ítems únicos disponibles en el catálogo aparecieron al menos una vez en alguna recomendación?
- Si tu cobertura es baja (ej: < 10%), ¿qué riesgo estratégico corre el retailer a largo plazo en términos de descubrimiento de productos y saturación de usuarios?

5.3. RecSys: Re-ranking Híbrido por Margen Financiero (profit-aware RecSys)

En el mundo real, maximizar el match con el usuario no siempre maximiza la utilidad de la empresa.

- Datos: Utiliza el archivo `movie_metadata.csv` (ver anexo) que contiene el `margin_category` (High/Low) para cada película.
- Tarea: Toma las predicciones del modelo ALS (scores de preferencia bruta). Implementa una función de re-ranking que multiplique el score original por un factor de peso según el margen:
 - Si `margin_category == "High"` → `Score_Final = Score_Original * 1.2`
 - Si `margin_category == "Low"` → `Score_Final = Score_Original * 0.9`
- Evaluación: Compara el Top-10 original vs. el Top-10 re-ranqueado para un usuario. ¿Cuántos ítems cambiaron?
- Dilema ético y estratégico: Discute el impacto de esta estrategia en la confianza del usuario vs. la rentabilidad de corto plazo. ¿Es sostenible "empujar" productos caros si son levemente menos relevantes?

Anexo de datasets

Files produced

- `retail\_reviews.csv` – NLP sentiment dataset (text reviews with labels)
- `video\_ratings.csv` – Explicit feedback ratings (user-movie ratings on 1-5 scale)
- `movie\_metadata.csv` – Movie dimension/metadata (profit margin, popularity)
- `music\_logs.csv` – Implicit feedback (user-song play counts)

Schema summary

`retail\_reviews.csv` (NLP Sentiment Dataset)

****Purpose:**** Text classification and sentiment analysis exercises.

Column	Type	Description
-----	-----	-----
`review_id`	int	Unique review identifier
`text`	string	Review text content
`sentiment`	string	Sentiment label

`video\_ratings.csv` (Explicit Feedback Ratings)

****Purpose:**** Explicit collaborative filtering, rating prediction, and recommender system evaluation.

Column	Type	Description
-----	-----	-----
`user_id`	int	User identifier
`movie_id`	int	Movie identifier
`rating`	int	User's rating for the movie

****Expected ranges:****

- `user\_id`: 0-599
- `movie\_id`: 0-149
- `rating`: 1-5

****Shape:**** ~20,000 rows

`movie\_metadata.csv` (Movie Dimension)

****Purpose:**** Movie attributes for filtering, stratification, and contextual features in recommendation tasks.

Column	Type	Description
`movie_id`	int	Movie identifier
`margin_category`	string	Profit margin classification
`popularity_index`	int	Popularity score

****Expected values:****

- `margin\_category`: "High" or "Low"
- `popularity\_index`: 1-100

****Shape:**** 150 rows (one per movie)

`music\_logs.csv` (Implicit Feedback)

****Purpose:**** Implicit feedback modeling (play counts as engagement proxy), session analysis, and implicit recommendation systems.

Column	Type	Description
`user_id`	int	User identifier
`song_id`	int	Song identifier
`play_count`	int	Total plays of song by user (aggregated)

****Expected ranges:****

- `user\_id`: 0-999
- `song\_id`: 0-299
- `play\_count`: ≥ 1

****Shape:**** ~40,000 rows (aggregated by user-song pairs)